HDOC Day-57

****

Write a C program that reads the size and elements of a variable-length integer array, arr[]. Pass the array to a function named max_subarray_sum() using its address. Use pointers to pass the array between the calling function and max_subarray_sum(). The function max_subarray_sum() must:

• Find the maximum sum of any contiguous subarray in arr[] using Kadane's algorithm.

• Display the maximum subarray sum.

• If all elements are negative, display the largest, or least negative, element.

• Ensure that the selected subarray contains at least one element.

1. Problem Understanding

The program reads the size of a variable-length integer array and its elements. The array is passed to max_subarray_sum() using its address. Kadane's algorithm is used to find the maximum sum among all contiguous subarrays. By initializing the current and maximum sums with the first element, the program also correctly handles arrays containing only negative values.

2. Standard Test Cases

Test Case	Size (n)	Array Input	Expected Output
1	9	-2 1 -3 4 -1 2 1 -5 4	Maximum subarray sum: 6
2	5	-8 -3 -6 -2 -5	Maximum subarray sum: -2
3	4	5 -2 3 4	Maximum subarray sum: 10
4	5	1 2 3 4 5	Maximum subarray sum: 15
5	4	-4 2 -1 3	Maximum subarray sum: 4

3. Algorithm

Main Algorithm

1. Start.

2. Read the size n.

3. Declare the variable-length array arr[n].

4. Read n array elements.

5. Pass arr and n to max_subarray_sum().

6. Stop.

Function: max_subarray_sum()

1. Set current_sum and max_sum equal to the first array element.

2. Traverse the array from the second element to the last element.

3. For each element, choose the larger value between the current element and current_sum + current element. Store it in current_sum.

4. If current_sum is greater than max_sum, update max_sum.

5. Display max_sum after the traversal is complete.

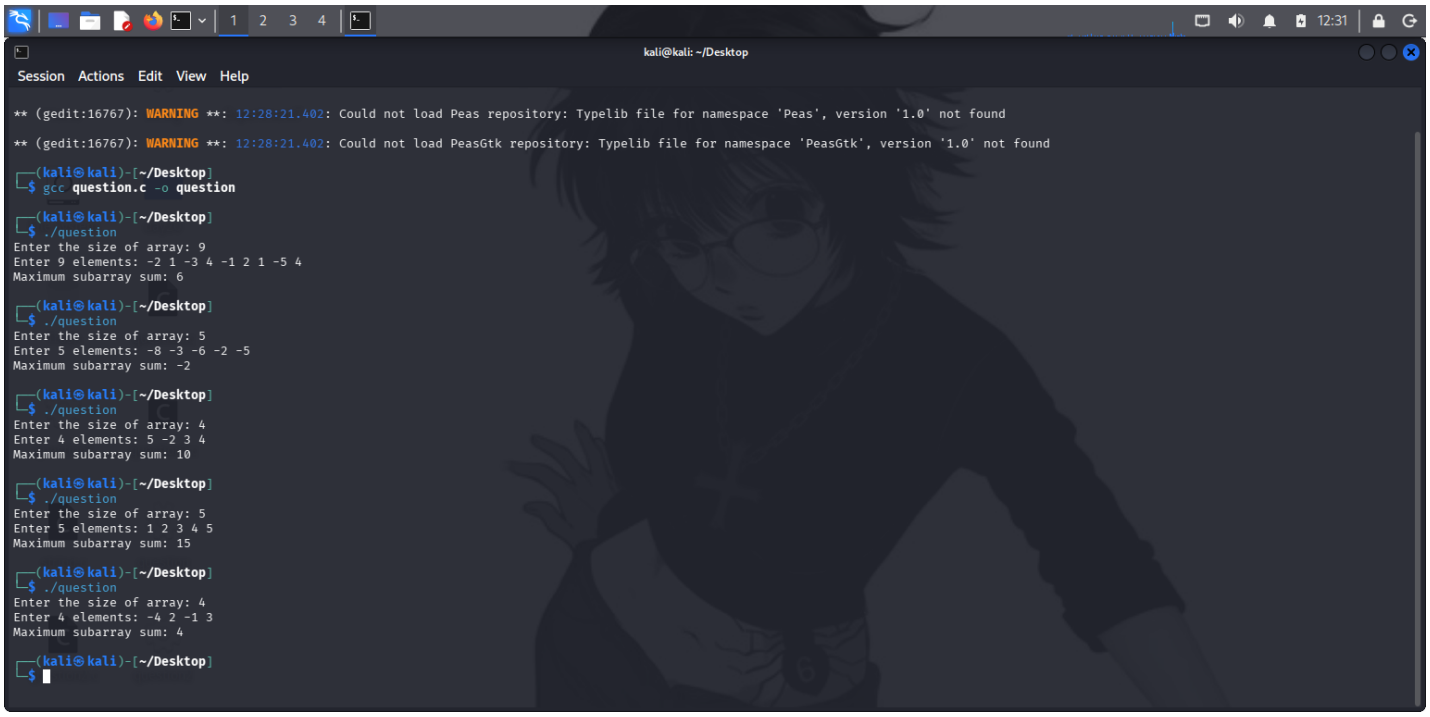
6. Since the first element is used for initialization, an all-negative array correctly returns its largest element.

4. C Program

```
#include <stdio.h>
void max_subarray_sum(int *arr, int n)
{
  int i;
  int current_sum = *arr;
  int max_sum = *arr;
  for(i = 1; i < n; i++)
  {
    if(*(arr + i) > current_sum + *(arr + i))
      current_sum = *(arr + i);
    else
      current_sum = current_sum + *(arr + i);
    if(current_sum > max_sum)
      max_sum = current_sum;
  }
  printf("Maximum subarray sum: %d\n", max_sum);
}
int main()
{
  int n, i;
  printf("Enter the size of array: ");
  scanf("%d", &n);
  int arr[n];
```

```
    printf("Enter %d elements: ", n);
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    max_subarray_sum(arr, n);
    return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	Actual Result	Status
1	9	-2 1 -3 4 -1 2 1 -5 4	Maximum subarray sum: 6	Pass
2	5	-8 -3 -6 -2 -5	Maximum subarray sum: -2	Pass
3	4	5 -2 3 4	Maximum subarray sum: 10	Pass
4	5	1 2 3 4 5	Maximum subarray sum: 15	Pass
5	4	-4 2 -1 3	Maximum subarray sum: 4	Pass